sensei-sql

Distilling a 9B-Parameter Language Model into a 1.5B Student for Offline Text-to-SQL

Slok Tulsyan (12342080), Vasu Garg (12342300), Krishna Jhawar (12341250), Siddharth Rai (12342050), Gaurav Kumar (12340790)

November 13, 2026

Abstract

Large language models translate natural-language questions into SQL well, but a 9B-parameter model needs about 15 GB of GPU memory and roughly one second per query. We study whether most of that ability can be transferred to a much smaller model. **sensei-sql** uses **Qwen3.5-9B** (FP8) as a *teacher* that writes and answers about 50k question/SQL pairs over the Spider training databases. Only pairs whose SQL executes and is confirmed correct (by self-consistency or by agreement with Spider's gold SQL) are kept, leaving 23,703 pairs. A **Qwen2.5-Coder-1.5B-Instruct** *student* is then fine-tuned on this data with QLoRA, alone and mixed with Spider's human-written data. On the Spider dev set (1,034 questions, official test-suite execution accuracy) the best student reaches **77.3%** against the teacher's **78.2%**, i.e. **98.8% of the teacher's accuracy**, while being 4.6× faster at batch size 1, 10.6× higher in throughput, 2.4× lighter in GPU memory and 4.5× smaller on disk. A 4-bit GGUF version (0.92 GiB) answers in 0.67 s median on a desktop CPU, 14.6× faster than the equally quantised teacher. We report two ablations (training-data source and amount), an error analysis, and a fully offline web application built around the model.

Contents

1 Introduction **3**

2 Background and Related Work **3**

3 Method **4**
3.1 Prompting and Safe Execution 4
3.2 Teacher Data Generation 4
3.3 Student Fine-Tuning (QLoRA) 5
3.4 Export and Deployment Formats 6

4 Experimental Setup **6**

5 Results **7**
5.1 Main Results 7
5.2 How Much of the Teacher Is Retained? 7
5.3 Accuracy by Hardness 8
5.4 CPU Deployment 8
5.5 Training Cost 9

6 Ablation Studies **10**
6.1 Ablation 1: Training-Data Source 10
6.2 Ablation 2: Amount of Teacher Data 11
6.3 Other Comparisons 11

7 Error Analysis **11**
7.1 Student versus Teacher, Question by Question 12

8 Deployment: The Offline Web Application **13**

9 Discussion and Limitations **13**

10 Conclusion and Future Work **14**

A Challenges and Design Decisions **14**
A.1 Problems and Fixes 14
A.2 Key Design Decisions 14

B Timeline and Compute **15**

C Reproduction **16**

1 Introduction

Text-to-SQL means converting a question such as "Which singers are older than the average age?" into an executable SQL query over a given database. It lets non-programmers query data directly, and recent large language models (LLMs) perform it well. The cost is deployment: the teacher used in this project (Qwen3.5-9B, FP8) needs about 15 GB of GPU memory and about 1 s per question, which is unattractive for laptops, on-premise databases, or settings where data must not leave the machine.

Knowledge distillation [5] offers a way out: a large teacher produces training signal for a small student. Here the teacher's signal is *data*: new question/SQL pairs for each training database, plus its own SQL for Spider's existing training questions. Because SQL can be executed, the data can be *verified* before the student ever sees it.

Contributions.

1. An execution-verified distillation pipeline that turns a 9B teacher into 23,703 clean training pairs over 146 training databases, with a strict guarantee that no dev database is touched.
2. A QLoRA-trained 1.5B student that retains **98.8%** of the teacher's Spider-dev execution accuracy (0.773 vs. 0.782), and a merged/quantised deployment path down to a 0.92 GiB GGUF file.
3. Two ablations—training-data source and amount of teacher data—plus an error analysis of the student's failures.
4. An offline web application (FastAPI back end, React front end) with four interchangeable inference back ends and a safe, read-only SQL runner.

A note on the headline number. The "99%" figure quoted for this project is a *relative* measure: the student's execution accuracy divided by the teacher's ($0.7727/0.7824 = 98.8\%$). In absolute terms the student answers 77.3% of Spider-dev questions correctly. We use the relative wording throughout.

2 Background and Related Work

Spider and execution accuracy. Spider [1] is the standard cross-domain text-to-SQL benchmark. Its dev databases are disjoint from the training databases, so it measures generalisation to unseen schemas. Many different SQL strings are correct for one question, so we use *execution accuracy* (EX): a prediction is right if it returns the same result as the gold query. We use the test-suite variant [2] that evaluates each query on several database copies to reduce false positives. Queries are grouped by Spider's hardness levels: easy, medium, hard, extra-hard.

Parameter-efficient fine-tuning. LoRA [3] trains small low-rank adapters on top of frozen weights; QLoRA [4] additionally keeps the frozen base model in 4-bit precision, making fine-tuning feasible on a single 20 GB GPU.

Distillation and self-consistency. Classic distillation [5] matches a student to a teacher's output distribution. Here we distil through generated *data*. Because synthesised questions have no gold answer, we use self-consistency [6] as a correctness proxy: a query is trusted if independent samples agree on its *execution result*.

Models and tooling. The teacher is Qwen3.5-9B (FP8-dynamic checkpoint by RedHatAI); the student is Qwen2.5-Coder-1.5B-Instruct [7]. Serving uses vLLM [8]; CPU deployment uses llama.cpp (GGUF) and Ollama.

3 Method

Figure 1 gives an overview. Every stage shares one prompt format, one schema formatter and one safe SQL executor, so there is no train/test mismatch.

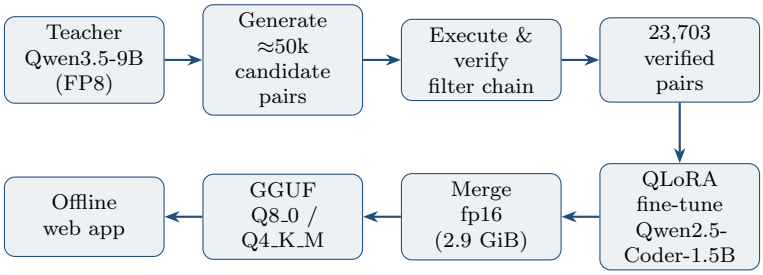


Figure 1: The sensei-sql pipeline: teacher data generation, execution-based verification, student fine-tuning, and deployment.

3.1 Prompting and Safe Execution

Schema text. A deterministic formatter renders each SQLite database as `CREATE TABLE` statements (tables sorted by name, columns in declared order, primary and foreign keys) plus three example rows per table, values truncated to 40 characters.

Prompt. One system message ("You are an expert SQLite developer… Reply with the SQL query only") and one user message ("Database schema: … Question: …"). The training target is the bare SQL string. The student is always used zero-shot.

Safe execution. Model-written SQL is never run unchecked: (1) `sqlglot` must parse exactly one `SELECT`/`WITH`/set-operation statement, so DDL, DML, `PRAGMA` and `ATTACH` are rejected before touching a database; (2) the connection is read-only (`mode=ro` plus `PRAGMA query_only`); (3) a wall-clock timeout and a row cap apply. The same runner is used for data filtering, scoring and the app. Two results match if they contain the same rows, ignoring row order unless the query has `ORDER BY`, and ignoring column names.

3.2 Teacher Data Generation

Only training databases (`train_spider` and `train_others`, 146 databases) are used; an assertion aborts the run if any dev database appears, which rules out leakage into the benchmark.

Two sources.

1. *Synthesis.* For each database and each of six SQL patterns (single-table filters; aggregation with grouping; joins; nested subqueries; `ORDER BY`/`LIMIT`; set operations) the teacher is asked for 10 question/SQL pairs, five times per pattern (temperature 0.9). This yielded 41,055 parseable candidates.
2. *Relabelling.* The teacher answers Spider's own 8,659 training questions greedily, giving teacher-written SQL for human-written questions.

The teacher runs with thinking disabled (`enable_thinking: false`), so it answers directly, which is the output format the student must imitate.

Filter chain. Every pair must pass all stages in order (Table 1).

Stage	Check	Synth. (kept)	Relabel (kept)
Input	candidate pairs	41,055	8,659
Parse	one valid `SELECT`	40,797	8,656
Execute	runs read-only in $\leq$5 s	39,041	8,388
Non-empty	result not empty/NULL; $\leq$5% empty per DB	33,236	6,839
Verify	synth: $\geq$3 of 5 re-samples reproduce the result; relabel: matches gold	18,986 (57%)	5,410 (79%)
Dedup	unique question per DB; $\leq$2 questions per SQL	18,346	5,357

Table 1: Teacher-data filter chain. Final dataset: 18,346 + 5,357 = 23,703 pairs (16–245 per database, median 170).

Self-consistency is the key quality gate for synthesised data: with no gold answer, agreement between independent samples is the proxy for correctness. The cap on empty results exists because "return nothing" queries are trivially easy to get right and teach little. The full run took about 7.5 h; enabling prefix caching sped up the five-fold re-sampling about 1.7×.

The kept pairs by pattern are: single-table 3,672; join 3,382; nested 3,220; set operations 2,961; aggregation 2,578; order/limit 2,533; relabelled 5,357. By re-derived Spider hardness: easy 3,957, medium 6,483, hard 6,506, extra 4,033, unparsed 2,724. A representative synthesised pair:

```
db: coffee_shop (nested, consistency 1.0)
Q: Find the names of members who have participated in happy hours
   at shops located in Hartford.
SQL: SELECT DISTINCT m.Name FROM member m WHERE m.Member_ID IN (
       SELECT Member_ID FROM happy_hour_member WHERE HH_ID IN (
         SELECT HH_ID FROM happy_hour h JOIN shop s ON h.Shop_ID = s.Shop_ID
         WHERE s.Address = 'Hartford'))
```

3.3 Student Fine-Tuning (QLoRA)

Item	Setting
Student	`Qwen/Qwen2.5-Coder-1.5B-Instruct`
Quantisation	4-bit NF4, double quantisation, bf16 compute (bitsandbytes)
LoRA	rank 16, $\alpha = 32$, dropout 0.05, all linear layers
Trainer	TRL `SFTTrainer`, prompt/completion format, loss on the SQL only
Optimisation	2 epochs, lr 2×10^{-4}, cosine schedule, 3% warm-up, weight decay 0
Batching	per-device 8 × grad. accumulation 2 (effective 16), `group_by_length`
Sequence length	max 4,096 tokens; longer examples dropped, not truncated (<1%)
Validation	2% held out from training databases; lowest-val-loss adapter kept
Seed	0 for every run

Table 2: Fine-tuning hyper-parameters, shared by all variants.

Three variants share all settings and differ only in data: (a) *human* (Spider train, 8,406 examples), (b) *teacher* (23,149), (c) *combined* (31,552). Two further ablation settings keep only 25% or 50% of the teacher pairs (human data is always kept whole in the combined variant). `group_by_length` matters: prompts range from about 300 to 4,096 tokens, and random batches wasted most compute on padding; grouping similar lengths cut step time from about 12 s to about 5 s.

3.4 Export and Deployment Formats

The best adapter is merged into the fp16 base weights (2.9 GiB) and verified: greedy SQL on the first 50 dev questions matches between adapter+base and merged model in 49 of 50 cases (the difference being identifier case or spacing, with identical execution results). The merged model is then converted with llama.cpp to GGUF and quantised to Q8_0 (1.53 GiB) and Q4_K_M (0.92 GiB).

4 Experimental Setup

Benchmark. Spider dev: 1,034 questions over 20 databases never used in training or data generation.

Metrics. *Accuracy*: official test-suite scorer (`taoyds/test-suite-sql-eval`), `etype=exec`. Because that scorer runs SQL on a writable connection without a real timeout, each prediction first passes through the safe runner; rejected, failing or timed-out predictions score 0. *Latency*: batch size 1, mean/p50/p95. *Throughput*: a second pass at batch size 32 (queries/s). *Peak VRAM*: device memory above the pre-load level (NVML), with a fixed 2 GiB KV cache for every model so numbers compare models rather than vLLM pre-allocation. *Size*: model directory (+ adapter).

Decoding. Greedy (temperature 0), maximum 256 new tokens, fixed seed; all GPU runs use vLLM 0.19.1.

Hardware. One NVIDIA RTX 4000 Ada (20 GB), driver 570 / CUDA 12.8, 31 GB RAM, Intel i9-14900K (32 threads). CPU runs use the app back ends on the first 100 dev questions at batch size 1.

Baselines. Teacher zero-shot and 3-shot (three training-database examples); untuned student zero-shot.

Why an FP8 teacher. Qwen3.5-9B in bf16 needs about 19 GB, leaving no room for a KV cache on a 20 GB card. The FP8-dynamic checkpoint runs natively on Ada GPUs (weights about 11.9 GiB when loaded text-only).

Statistical caution. With 1,034 questions the standard error of an accuracy near 0.77 is about ±1.3 points, and each variant was trained once. Differences below roughly 2.5 points are therefore treated as unproven.

5 Results

5.1 Main Results

Model	EX	Easy	Med.	Hard	Extra	p50 (ms)	q/s (bs 32)	VRAM (GiB)	Size (GiB)
Teacher Qwen3.5-9B, 0-shot	**.782**	.940	.834	.707	.488	945	4.2	15.5	13.1
Teacher, 3-shot	.775	.944	.848	.667	.440	1252	1.7	16.0	13.1
Student, no fine-tuning	.672	.867	.729	.511	.398	214	42.7	6.5	2.9
Student (a) human only†	.755	.907	.812	.632	.506	224	38.7	7.2	3.0
Student (b) teacher only†	.769	.911	.834	.713	.440	244	42.4	7.2	3.0
Student (c) human + teacher†	.772	.923	.807	.707	.518	235	39.0	7.2	3.0
Student (c) merged fp16 (final)	**.773**	.919	.812	.713	.512	**206**	**44.1**	**6.5**	**2.9**

Table 3: Spider dev (1,034 questions), RTX 4000 Ada, vLLM. †LoRA adapter served through vLLM, which adds some latency and memory; merging removes this overhead.

Few-shot prompting did not help the teacher (0.775 vs. 0.782), so zero-shot was used throughout. Figure 2 shows the central trade-off: the fine-tuned students sit far to the left of the teacher (much faster) at almost the same accuracy.

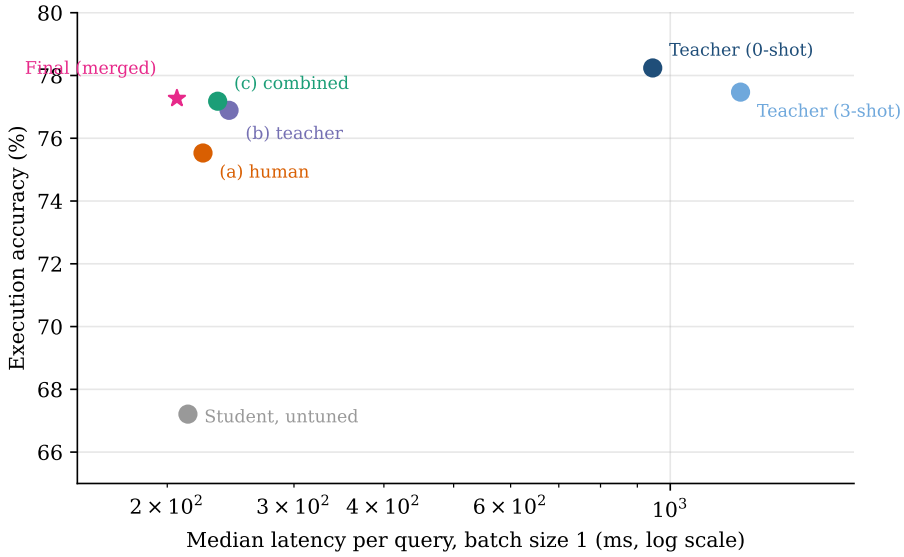


Figure 2: Accuracy versus median latency on GPU (log scale). The merged final student (star) is the fastest model and within one point of the teacher.

5.2 How Much of the Teacher Is Retained?

	Teacher	Student (final)	Ratio
Execution accuracy	0.782	0.773	**98.8%** retained
Median latency (batch 1)	945 ms	206 ms	4.6× faster
Throughput (batch 32)	4.2 q/s	44.1 q/s	10.6×
Peak GPU memory	15.5 GiB	6.5 GiB	2.4× less
Size on disk	13.1 GiB	2.9 GiB	4.5× smaller

Table 4: Final merged student versus teacher.

Distillation closed about 92% of the gap between the untuned student and the teacher: $(0.773 - 0.672)/(0.782 - 0.672) \approx 0.92$.

5.3 Accuracy by Hardness

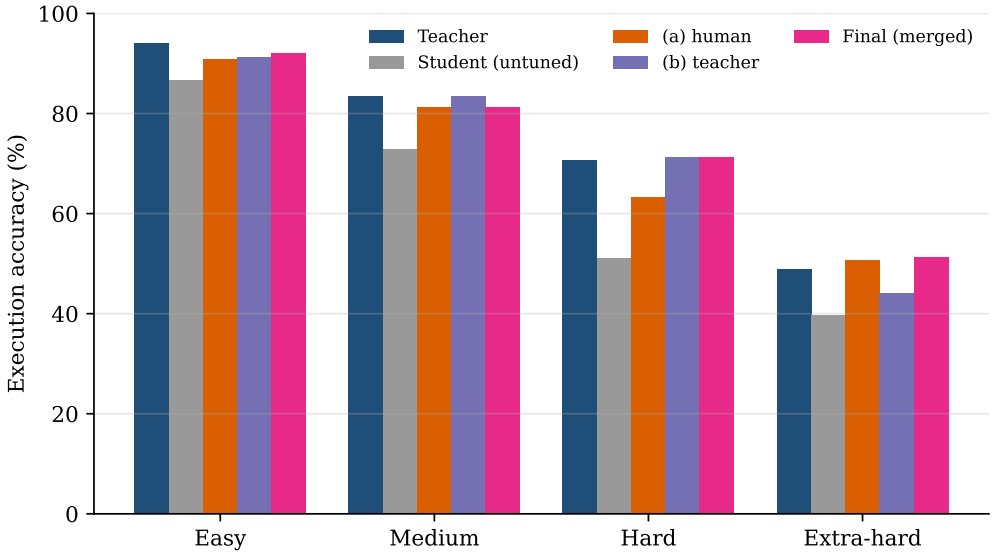


Figure 3: Execution accuracy by Spider hardness level.

Compared with the untuned student, the final model improves hard questions from 0.51 to 0.71 and extra-hard from 0.40 to 0.51; on easy questions (0.87→0.92) the base model was already close. On extra-hard questions it matches the teacher (0.512 vs. 0.488, within noise). All models degrade with hardness, and extra-hard questions remain the main gap in absolute terms, for the teacher as well.

5.4 CPU Deployment

Model	Format / runtime	Size (GiB)	p50	p95	EX (n=100)
Student	fp32, transformers	2.89	5.98 s	10.6 s	0.83
Student	Q8_0, llama.cpp	1.53	1.24 s	5.09 s	0.80
Student	Q4_K_M, llama.cpp	**0.92**	**0.67 s**	4.86 s	0.77
Teacher	Q8_0, llama.cpp	9.11	13.7 s	23.5 s	0.84
Teacher	Q4_K_M, llama.cpp	5.38	9.86 s	18.8 s	0.84

Table 5: CPU results (Intel i9-14900K, 32 threads, batch size 1, first 100 dev questions). These 100 questions are easier than average, so accuracies are comparable with each other only, not with Table 3.

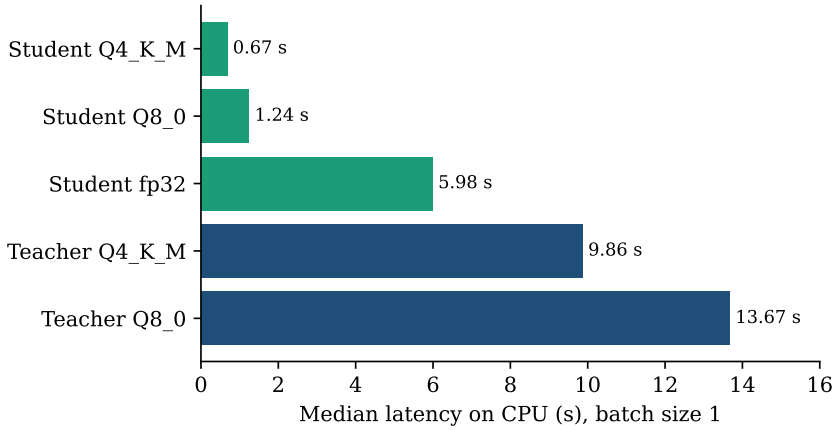


Figure 4: Median CPU latency per query. The student is 14.6× faster than the teacher at Q4_K_M and 11× faster at Q8_0.

On the same machine, runtime and quantisation, the student is 14.6× faster than the teacher at Q4_K_M and 11× faster at Q8_0, and 5.8× smaller at Q4_K_M. Its Q4_K_M file is 8.9× faster than its own fp32 version. On these 100 questions the teacher scores 0.84 in both formats and the student 0.80 (Q8_0) and 0.77 (Q4_K_M), i.e. 95% and 92% of the teacher. The gap is larger than on full dev; the standard error at $n = 100$ is about ±4 points, so it may be partly noise and partly the cost of 4/8-bit quantisation, which this experiment does not separate. The teacher GGUF was converted from the FP8 checkpoint (dequantised, then re-quantised), so it is not exactly the model used in the GPU runs.

Quantised models often write the same SQL with different case or quote style, so text identity is a poor test: against the fp32 student only 47 (Q8_0) and 29 (Q4_K_M) of 100 outputs are textually identical, while correctness agreed on 91 and 86 questions.

5.5 Training Cost

Run	Train examples	Steps	Minutes	Best val. loss
Human	8,406	1,052	94	0.065
Teacher	23,149	2,894	250	0.106
Combined	31,552	3,944	350	0.114
Teacher 25%	5,786	724	64	0.131
Teacher 50%	11,566	1,446	125	0.119
Combined, 25% teacher	14,185	1,774	161	0.136
Combined, 50% teacher	19,973	2,498	225	0.123

Table 6: Training runs (peak memory 8.3–8.4 GiB each). Validation losses are measured on each variant's own held-out data and are *not* comparable across variants.

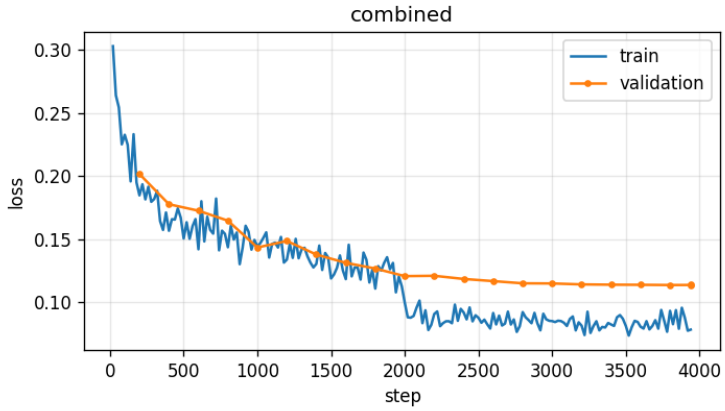


Figure 5: Training and validation loss of the combined run. Validation loss flattens after about 2,500 steps; the best checkpoint is kept.

6 Ablation Studies

Two questions: (1) does teacher-generated data help compared with Spider's human labels? (2) how much teacher data is needed? Only the data changes between runs.

6.1 Ablation 1: Training-Data Source

Training data	Examples	EX	Easy	Med.	Hard	Extra	Gain
None (untuned)	0	.672	.867	.729	.511	.398	–
(a) Human only	8,406	.755	.907	.812	.632	.506	+8.3
(b) Teacher only	23,149	.769	.911	.834	.713	.440	+9.7
(c) Human + teacher	31,552	.772	.923	.807	.707	.518	+10.0

Table 7: Effect of the training-data source (gain in points over the untuned student).

- Every kind of fine-tuning helps substantially (+8 to +10 points).
- Teacher data alone matches or beats human labels (0.769 vs. 0.755), although no human wrote it. The 1.4-point gap is about one standard error, so the defensible claim is "at least as good", not "better". The teacher set is also 2.8× larger, so source and size are confounded.
- Adding human data to teacher data gives a further +0.3 points, which is not significant overall.
- The hardness breakdown is more informative: teacher data helps most on *hard* questions (0.713 vs. 0.632), human data helps most on *extra-hard* (0.506 vs. 0.440), and the combination gets both (0.707 and 0.518). This is the best reason to prefer variant (c). A plausible, untested explanation: synthetic data covers nested queries and set operations evenly, whereas Spider's human questions contain more long, awkward multi-table phrasings.

6.2 Ablation 2: Amount of Teacher Data

Teacher pairs used	Teacher only		Human + teacher	
	Examples	EX	Examples	EX
25%	5,786	0.750	14,185	0.767
50%	11,566	0.760	19,973	0.759
100%	23,149	0.769	31,552	0.772

Table 8: Effect of the amount of teacher data.

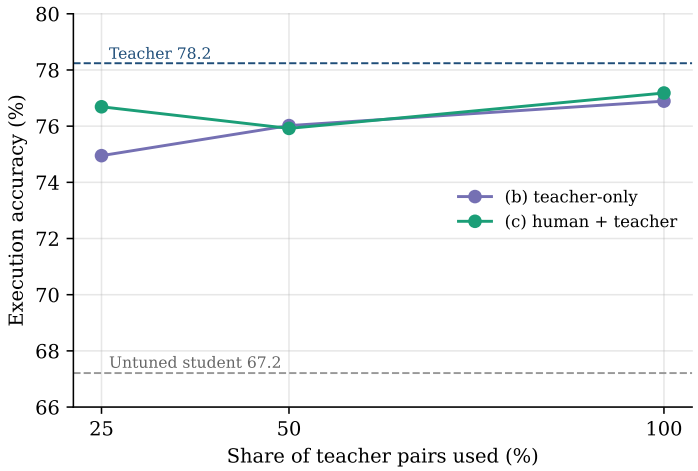


Figure 6: Execution accuracy versus share of teacher pairs used.

Teacher-only accuracy rises steadily (about +1 point per doubling) with no sign of saturation, so more verified teacher data would probably help. The combined variant is flat within noise (0.759–0.772): with human data already present, 25% of the teacher pairs (about 5,800) reaches 0.767, within about 0.5 points of the full set. The dip at 50% is about one standard error and should not be read as a real effect. Practically, if generation is expensive, human data plus a quarter of the teacher pairs recovers almost everything.

6.3 Other Comparisons

- **Few-shot vs. zero-shot (teacher):** three examples did not help (0.775 vs. 0.782) and were 1.3× slower.
- **Merged vs. adapter (same weights):** 0.773 vs. 0.772; merging loses nothing and removes adapter overhead.
- **Quantisation (CPU subset):** fp32 0.83, Q8_0 0.80, Q4_K_M 0.77 on 100 questions (Table 5).

7 Error Analysis

We analysed the best student before merging, variant (c), which makes 236 errors on 1,034 questions. One hundred failures were sampled at random (seed 0) and categorised by a heuristic comparing the tables, columns and clauses used in the predicted and gold SQL. The labels are approximate.

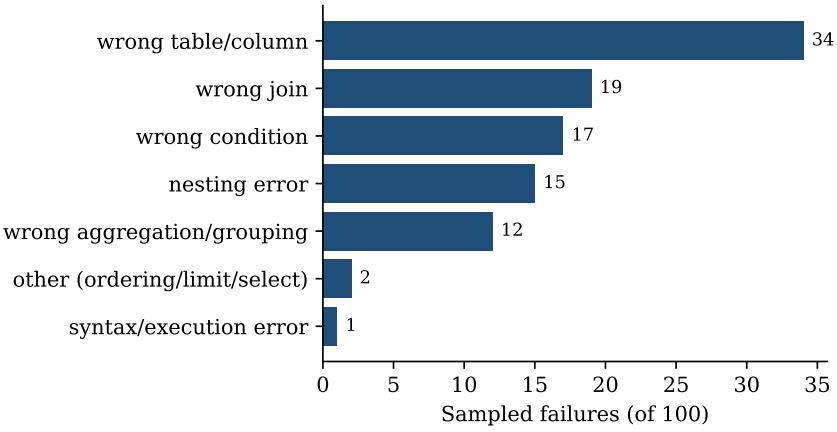


Figure 7: Categories of 100 sampled failures of the best student.

- **The student almost always writes valid SQL.** Only 1% of sampled failures did not execute; the rest run and answer a slightly different question.
- **Schema linking is the commonest mistake** (wrong table/column 34%, wrong join 19%). For "the name and rank of the 3 youngest winners" the student joined `players` and `matches`, when columns of `matches` alone sufficed.
- **Conditions** (17%): for "nations considered US territory" it filtered `GovernmentForm = 'US'` instead of `'US Territory'`.
- **Nesting and set operations** (15%) fail mostly on hard/extra questions: "countries not playing any cartoon by Todd Casey" needs `EXCEPT` or `NOT IN`, and the student wrote a `!=` filter.
- **Aggregation/grouping** (12%) and **other** (2%) account for the rest.
- **Some failures are benchmark artefacts**: case-sensitive string matching against a lower-cased gold literal, tie-breaking in `ORDER BY ...LIMIT 1`, and ambiguous questions where either reading is reasonable. Execution accuracy is strict, so the true error rate is somewhat below 22.8%.

7.1 Student versus Teacher, Question by Question

Outcome	Questions
Both correct	706
Student correct, teacher wrong	92
Teacher correct, student wrong	103
Both wrong	133

Table 9: Final student vs. teacher on 1,034 dev questions (model (c)).

The two models fail on largely different questions, consistent with their near-equal accuracy. When the student wins, the teacher typically adds an unnecessary join or picks the wrong filter; when the teacher wins, the student invents a join or aggregates the wrong column.

8 Deployment: The Offline Web Application

The distilled student powers a local, fully offline demo. The back end is a FastAPI service exposing:

Endpoint	Purpose
`GET /api/health`	active back end name and title
`GET /api/databases`	available databases (and uploaded ones)
`GET /api/schema?db=...`	schema text exactly as shown to the model
`POST /api/ask`	{db, question} → SQL, status, result table, timings
`POST /api/run`	run user-edited SQL through the same safe runner
`POST /api/upload`	register an uploaded `.sqlite` file (up to 200 MB)

The front end (React + TypeScript, built with Vite and shadcn/ui components) has two pages. The *Assistant* page lets the user pick a Spider database or upload a SQLite file, ask a question, and see the generated SQL (editable, with a Run button), the result table, the schema, timings and a query history. A *Results* page presents the evaluation tables and charts of this report.

Four interchangeable inference back ends share one interface: **vLLM** (GPU), **transformers** (GPU or CPU), **llama-server** (CPU, GGUF) and **Ollama**. Safety is built in: only a single `SELECT`/`WITH` statement is accepted, the database is opened read-only, and an edited `DELETE` is rejected. If a generated query fails, the error message is shown to the model once for a corrected attempt (`self_correct`). Offline flags (`HF_HUB_OFFLINE`, disabled analytics) are set, so after the one-time download nothing needs the network.

9 Discussion and Limitations

What is established. Fine-tuning helps by about 8–10 points, and the final student ends within one point of the teacher while being several times faster and smaller. These effects are larger than the measurement noise.

What is not.

- **One benchmark.** Spider dev is SQLite with moderate-size schemas; results may not carry over to other dialects, large schemas or messy real-world questions. BIRD and Spider 2.0 were not tried.
- **Statistical power and seeds.** Gaps below about 2.5 points (teacher-only vs. human-only, combined vs. teacher-only, most of the data-size curve) are not established; each setting used a single seed.
- **Execution accuracy only.** It can credit queries that match by chance and penalise correct queries on ambiguous questions.
- **The teacher is not a gold standard.** Its mistakes can enter the distilled data; filters check executability and consistency, not truth.
- **Data-source confound.** The teacher set is 2.8× larger than the human set; Ablation 2 only partly addresses this.
- **CPU numbers** use 100 easier-than-average questions on one desktop CPU, and quantisation loss is not separated from noise.
- **Prompt content.** The prompt includes sample rows of each table; its effect was not ablated.

Reproducibility. Evaluation decoding is greedy with fixed seeds. Batched GPU kernels can cause small numeric differences across machines, so expect accuracy to reproduce within a few questions rather than bit-for-bit; training is not bit-reproducible across hardware.

10 Conclusion and Future Work

A 1.5B-parameter student trained on execution-verified data from a 9B teacher reaches 77.3% execution accuracy on Spider dev, **98.8% of the teacher's 78.2%**, with 4.6× lower latency, 10.6× higher throughput, 2.4× less GPU memory and a 4.5× smaller footprint. Quantised to 4 bits, it fits in 0.92 GiB and answers in under a second on a CPU, making a private, offline text-to-SQL assistant practical.

Future work.

1. Repeat every setting with several seeds and report mean ± standard deviation.
2. Evaluate the GGUF models on the full dev set to separate quantisation loss from noise.
3. Generate more teacher data (the teacher-only curve had not saturated) and test stricter filters.
4. Train on error-and-fix pairs to improve self-correction.
5. Improve schema linking (the main error class) with column descriptions or retrieved values.
6. Evaluate on BIRD or an in-domain held-out set; try preference-style distillation.

A Challenges and Design Decisions

A.1 Problems and Fixes

Area	Problem	Fix
GPU stack	Default install pulled CUDA 13 wheels; the CUDA 12.8 driver could not run them	Pinned torch 2.10.0+cu128 and vLLM 0.19.1
Teacher memory	vLLM ran out of memory profiling CUDA graphs on 20 GB	Text-only loading, `max_num_seqs`=64, `max_num_batched_tokens`=4096
Teacher size	bf16 weights (∼19 GB) leave no KV cache	FP8-dynamic checkpoint (∼11.9 GiB)
Scoring safety	Official scorer has no effective timeout and runs arbitrary SQL	Every prediction first goes through the safe runner
Hardness labels	Spider's rules cannot parse some styles (implicit aliases)	Rewrite to Spider style; fall back to gold hardness
Training speed	12 s/step, mostly padding	`group_by_length`: ∼5 s/step
Post-train check	vLLM needed ∼17.6 GB but trainer still held ∼9.5 GB	`release_gpu()` before the check
Long jobs	Watcher script matched its own command line, missing the end of the 7.5 h run	Wait on the process ID
App shutdown	Orphaned `EngineCore` process held ∼7 GB GPU memory	Explicit `close()`; SIGTERM exits cleanly
CPU comparison	CPU subset compared against full GPU run	Blank ratio columns for subset runs

Table 10: Selected problems encountered and their fixes.

A.2 Key Design Decisions

- **Train only on training databases; assert no dev database**, so the headline result cannot be invalidated by leakage.

- **Verify teacher data by execution**, not trust: an unverified teacher would pass its own errors to the student.
- **Cap empty results at 5%; deduplicate** per database and per SQL.
- **Generate both new and relabelled questions**: synthesis broadens pattern coverage; relabelling tests whether teacher SQL for human questions is as good as human SQL.
- **Thinking disabled for the teacher**: direct answers are what the student must imitate.
- **Same prompt function everywhere**, greedy decoding, fixed seed, fixed 2 GiB KV cache for all models.
- **Loss on the completion only; drop rather than truncate** over-long examples.
- **Choose the variant by full-dev accuracy, not validation loss**, since losses are not comparable across variants.
- **Judge quantised models by accuracy, not text identity.**

Abandoned or unhelpful: few-shot prompting for the teacher; random-order batching; one interrupted ablation run (combined, 25% teacher), stopped at step 1,370 and rerun from scratch because only checkpoints, not an adapter, had been saved.

B Timeline and Compute

Stage	Wall-clock
Teacher data generation (146 DBs, all filters)	∼7.5 h
Baselines: teacher 0-shot / 3-shot / untuned student	25 / 35 / 5.5 min
Training: human / teacher / combined	94 / 250 / 350 min
Training: teacher 25% / 50%	64 / 125 min
Training: combined 25% / 50% teacher	161 / 225 min
Full-dev evaluation of one student	∼5–6 min
Merge + verify + merged eval	∼10 min
GGUF conversion and quantisation	∼1 min
Teacher GGUF conversion + CPU evals (Q4_K_M / Q8_0)	5 min + 18 / 25 min

Table 11: Wall-clock time per stage on one RTX 4000 Ada. Training totals about 21 h; the whole project took roughly 30 GPU-hours (excluding about 2 h lost to the interrupted run).

The project was developed in seven work packages (setup; shared core; evaluation harness and baselines; teacher data generation; QLoRA fine-tuning and export; final evaluation, ablations and error analysis; offline app), from 25 Oct to 11 Nov 2026. The code base is about 3,100 lines in five packages (`core`, `teacher`, `train`, `eval`, `app`) with 66 unit tests.

C Reproduction

Item	Version
Python / torch / vLLM	3.10.12 / 2.10.0+cu128 / 0.19.1
transformers / TRL / PEFT	5.20.2 / 1.14.2 / 0.21.2
bitsandbytes / Gradio / sqlglot	0.50.2 / 6.30.0 / 30.24.0
Teacher	`RedHatAI/Qwen3.5-9B-FP8-dynamic` @ `790f057`
Student	`Qwen/Qwen2.5-Coder-1.5B-Instruct` @ `2e1fd39`
Scorer	`taoyds/test-suite-sql-eval` @ `e97acc5`
llama.cpp	commit `620d222`, CPU-only build

Table 12: Pinned software and model versions (all seeds 0).

```
make install && make download # environment, Spider, models, scorer
make check && make test # setup check, 66 unit tests
make teacher # teacher data (~7.5 h)
python -m sensei_sql.train configs/train/combined.yaml
python -m sensei_sql.train.export merge --adapter models/adapters/combined
python -m sensei_sql.eval --model models/student-merged
python -m sensei_sql.train.export gguf
make results # tables, plots, error analysis
make app # offline web app
```

References

[1] T. Yu et al. *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task.* EMNLP, 2018.

[2] R. Zhong, T. Yu, D. Klein. *Semantic Evaluation for Text-to-SQL with Distilled Test Suites.* EMNLP, 2020.

[3] E. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models.* 2021.

[4] T. Dettmers et al. *QLoRA: Efficient Finetuning of Quantized LLMs.* 2023.

[5] G. Hinton, O. Vinyals, J. Dean. *Distilling the Knowledge in a Neural Network.* 2015.

[6] X. Wang et al. *Self-Consistency Improves Chain of Thought Reasoning in Language Models.* 2023.

[7] B. Hui et al. *Qwen2.5-Coder Technical Report.* 2024.

[8] W. Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention.* SOSP, 2023.